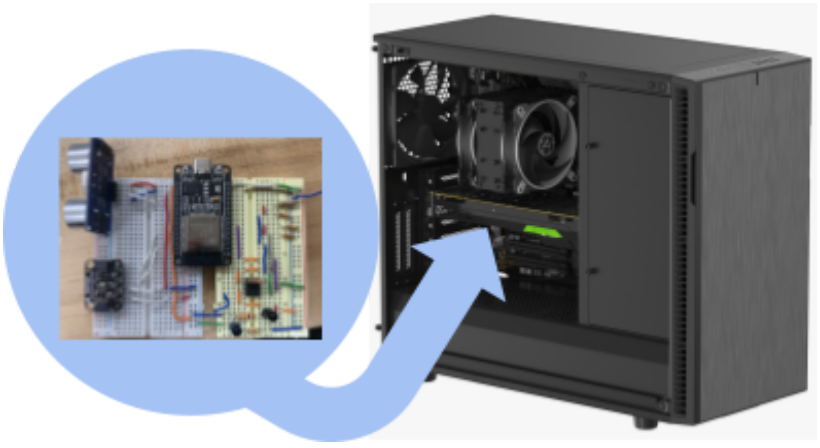
UNIVERSITY OF GEORGIA

Course Project - Fall 2024

DESKTOP AIR COOLING MONITOR

NATHAN PARK



Introduction and Motivation

The purpose of this project is to create a sensor that will monitor various factors of a desktop computer that would contribute to its air cooling. Specifically, the goal was to measure temperature, air flow, fan speed, and dust concentration.

- Regarding the air flow measurement, it is intended to use the BMP280's air pressure reading. The BMP280 will also be used for the temperature reading.
- For the fan speed, the HC-SR04 ultrasonic distance sensor measures the number of pulses within a given time.
- Finally, to measure dust concentration, an infrared photodiode alongside an infrared light source will measure scattered light, with the photodiode pointed towards a space in which the emitted IR light can be scattered by dust particles.

The device will be mounted inside the PC with the HC-SR04 pointed towards one of the case fans. Additionally, the device can be pointed towards the CPU cooler fan.

For PC performance, regulating temperature is extremely important, with many factors that contribute to it.

1. The coolers within the PC, such as the CPU cooler and case fans, should be performing optimally and at the necessary speed.
2. Because the fans constantly draw in cool air, dust often collects within the case, which often prevents optimal performance.
3. These fans should produce optimal air flow within the PC as well.

For these reasons, it would be best to measure the input factors such as fan speed and dust concentration and measure the resulting air flow and air temperature. This would give me a good idea of the PC's current physical state. Although the PC itself has a sensor for measuring temperature and a displayed fan speed, the sensors can be used to both confirm these measurements and elaborate upon the relationship between the input factors and measured output factors. The fan speed alone isn't enough to determine if the PC's temperature is being regulated properly because the air flow and dust concentration conditions greatly affect the effects of the fan. Additionally, when building a PC, monitoring air flow is very important to make sure that the PC's fans are mounted and directed properly.

Physical Environment

The physical environment plays an important role in the accurate measurement of all 3 sensors (BMP280, HC-SR04, photodiode). The setup will involve a 3D printed air chamber for measuring the dust concentration.

- The air chamber must be (almost) completely enclosed to ensure minimal light interference.
- The air chamber must have intake and outtake holes for the air containing dust particles to enter and exit the chamber
- The air chamber must include mounts for the light source and photodiode.

The entire device will be placed within the PC itself, with the HC-SR04 pointed towards a desired fan. Ideally, a fan will also be blowing towards the chamber to facilitate airflow within the air chamber itself.

The device will have three different areas to test its capabilities. There are two desktop computers that can be tested, with one being older (and more dusty) than the other. The third area could be any room (such as Driftmier 1450) as a control for air flow and dust concentration. It is expected that the new PC to have the greatest airflow, the old PC to have the highest dust concentration, and the room to have the lowest dust concentration and least air flow:

Conditions	Room 1450	Old PC	New PC
Temperature	Low	High	Middle
Fan Speed	n/a	High	Low
Air Flow	Low	Middle	High
Dust Concentration	Low	High	Middle

Figure 1: Environment Expectations

As shown by Figure 1, the expectation is that the older PC will have increased fan speeds due to worse air flow and temperature regulation.

See Appendix B for the General Safety Checklist.

Sensor Dynamics

BMP280 (Temperature and Air Flow)

The BMP280 prepackaged sensor will measure both the temperature and air flow.

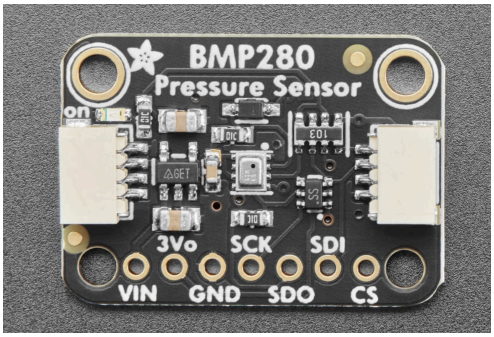


Figure 2: BMP280

To measure temperature, the BMP280 Arduino package makes it quite simple. However, for pressure, we can infer air flow using the air pressure sensor. Air flow is created through differential pressure (moving from high pressure to low pressure), meaning that if we read a nominal pressure from outside the desktop, the internal pressure will give us a good idea of the air flow.

HC-SR04 (Fan Speed)`

The HC-SR04 ultrasonic sensor will measure the speed of the fans. The distance will need to be calibrated in order to accurately calibrate whether the currently measured distance is on the blades of the fan (HIGH state) or behind the fan (LOW state).

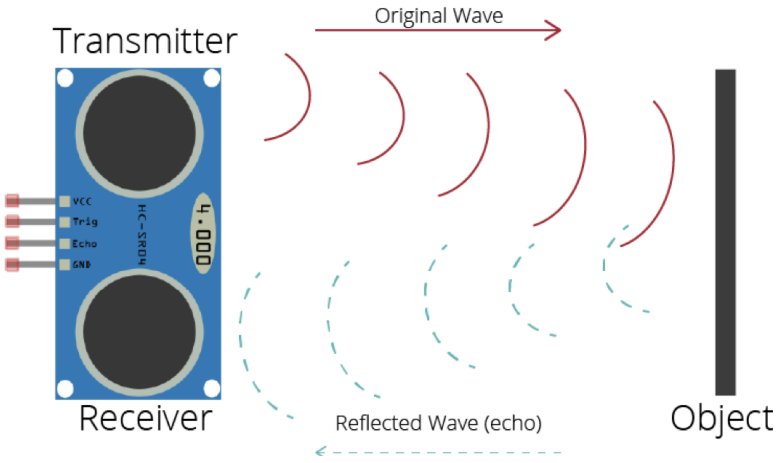


Figure 3: HC-SR04 Functionality

According to the data sheet, the resolution is 0.3cm. This resolution should be enough to distinguish whether the state is high or low. Therefore, all other values outside of the closest point on the blade of the fan are not considered. For instance, if the measurement at this point is 10 cm, any distances farther from this point (10.3cm or higher) will be considered a low state. Additionally, any measurements that are less than the 10cm (9.7cm or lower) should not be considered as actual measurements.

The number of rising edges (from low state to high state) in a given period of time divided by the number of blades on the fan should result in the fan speed. Still using the 10cm example, we can determine the maximum speed of the fans that can be read.

Finally, the theoretical sampling rate of the HC-SR04 is 344Hz. This means that it can measure up to 20,640RPM for all blades of the fan. However, after testing, this sampling rate could have been significantly lower than the theoretical estimate. If this was the case, then the VL53L0X laser distance measurer can be used instead since it has a much higher theoretical sampling rate due to it using light rather than sound. The concept of measuring the RPM of the fans with the new sensor would have been the same. However, the HC-SR04 provided sufficient sampling speeds to measure the fan.

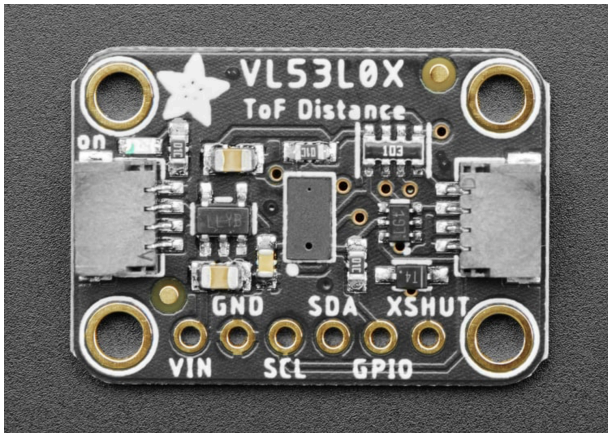


Figure 4: VL52L0X Laser Distance Measurer

Photodiode and IR LED (Dust Concentration)

The IR photodiode will measure dust concentration through the scattering of IR light emitted from an LED.

Figure 5: QSD2030F Photodiode

In this environment, the light will be scattered from dust particles in the air. The more light that is measured from the photodiode, the higher the dust concentration. For this to work, the photodiode must not be pointed directly towards the IR LED, since the direct emission would interfere with the measurement of the light intensity from the scattered light.

The signal from the photodiode will be noisy, so some filtering will need to be done in order to clean up the signal. Additionally, the changes in the photodiode output voltage will be small, so amplification will be needed to scale the signal properly.

Complete Signal Processing

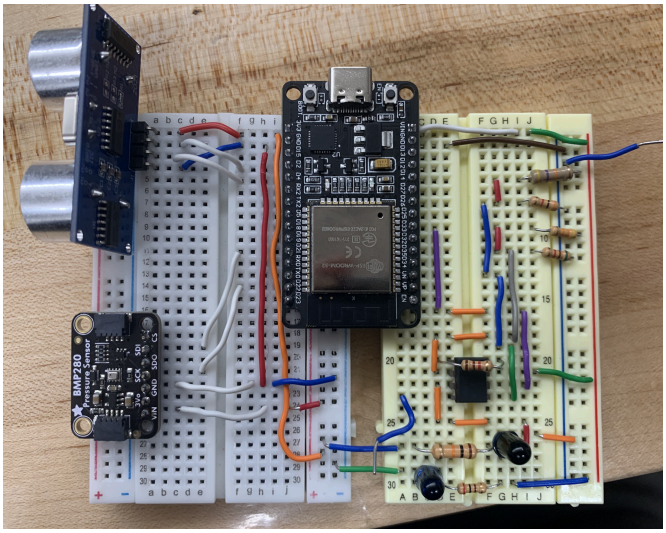


Figure 6: Complete Circuit

Power Supply

Throughout this project, I've tried to ensure that only the DC power source is the ESP-32, rather than an external power supply. However, because the instrumentation amplifier needed a negative voltage for the minus supply, I've needed to use the DC power supply to properly use the INA121P during amplification.

Analog Signal Processing

The analog signal from the photodiode that is measuring dust concentration is the result of the following setup (Figure 6, bottom right):

- An IR emitter diode connected to the 5V through a 200 Ohm resistor (for a 1.2V output), pointing straight upward.
- The IR photodiode connected to a 5.4K resistor that is from the 3.3V source, also pointing straight upward.
- Because the sensor is not pointing towards the emitter, the main source of IR light would be through when it scatters from dust (or other particles in the air). This configuration may change.

The output voltage from the photodiode ranges from 0.54V (no IR measurement) to 0.58V (full IR measurement).

Amplification Stages

There are three things that need to be done to the output voltage of the photodiode in order for it to be measured using the full range of the ADC (which makes full use of the resolution capabilities of the ADC). Shown in Figure 6, right side.

1. Decrease the minimum voltage to roughly 0V through an offset.
2. Amplify the 0.04V difference to be around a 2.7V difference.
3. Raise the voltage by 0.3V.

$$V_{\text{out}} = G_{\text{amp}}(V_{\text{in}} - V_{\text{offset}}) + V_{\text{bias}}$$

These three steps were done through an instrumentation amplifier with a differential amplifier configuration (theoretically). The in-amp is the INA121P.

1. Applying a 0.54V offset to pins 1 and 8 (OFFSET N1 and N2)
    a. The 0.54V is achieved from a voltage divider of 42.5kOms and 5.08kOhms, which ended up to be closer to 0.525V.
    b. Should result in the input voltage to the op-amp being decreased by 0.525V, making the range 0.15V to 0.55V.

2. Use the correct resistor value to achieve the intended gain.
    a. V+ connected to 3.3V (ESP-32), V- connected to -5V (external DC power supply)
    b. R_g = 1.1kOhms
    c. Should result in a 46.25 gain, which means that the output range 0.68V to 2.48V.
3. Applying a 0.3V bias through the reference pin of the INA121P.
    a. The 0.3V is obtained from a voltage divider with 47kOhms and 3kOHms, which ended up being closer to 0.281V.
    b. By applying to the reference pin of the amplifier, the output voltage is scaled by 0.281V making the final range 0.96V to 2.76V.

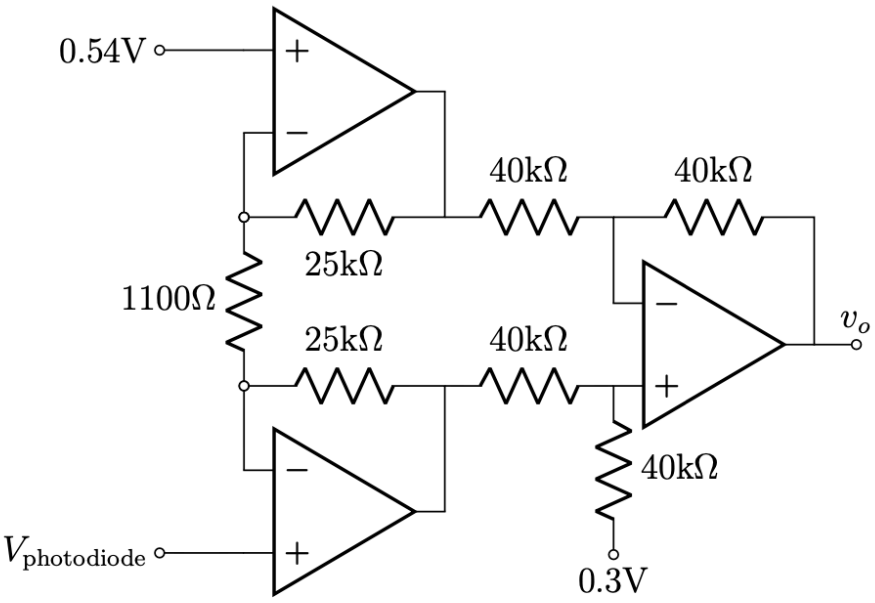


Figure 7: In-Amp Design

After amplification, the signal, which previously ranged 0.54V to 0.58V, should range from 0.96V to 2.76V, which is optimal for connecting to the ADC. Although it is not perfectly matched to be the ideal range of 0.3V to 2.7V, where the error was due to the difference in resistors within the voltage dividers, the final range still achieves a much more suited operating range to the ADC pins compared to the original photodiode output without amplification.

Analog to Digital Converter and Filtering Stage

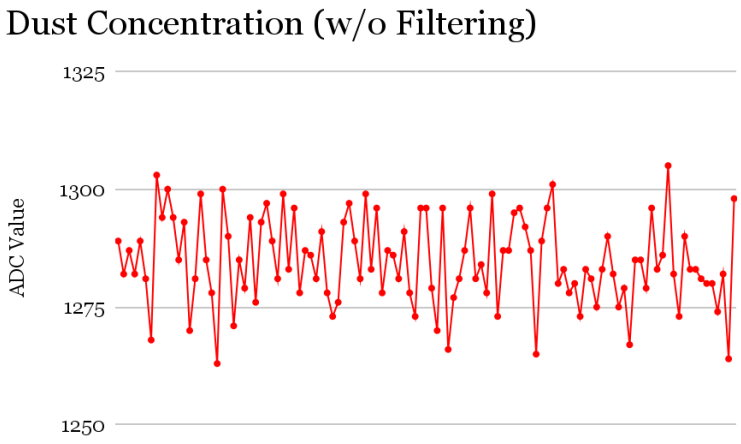


Figure 8: No Filtering Applied

Once the dust concentration voltage signal is an appropriate voltage value to read into the ADC pins of the ESP-32, connecting the output signal results in a waveform similar to that shown in Figure 8. Filtering is clearly needed and, although filtering is not implemented in hardware, the Kalman filter (done completely in software) should clean up the noisy signal from the raw sensor output measured from the photodiode. See Digital Logic Implementation for the full explanation of the implementation of Kalman filtering.

Digital Logic Implementation

HC-SR04

For this project, most of the digital logic was focused upon measuring fan speed using the HC-SR04. The author needed to see if it was possible to read the required changes in distance at a fast enough rate to measure up to 1000 RPM. Through testing of the HC-SR04.h package, the following conclusions were made:

- The theoretical sampling rate of ultrasonic sound is fast enough to measure the required speeds.
- The HC-SR04.h package does not take advantage of the theoretical sampling rate of the sensor so that it can measure distances that are farther away.

From these conclusions, the author decided to write a custom library to utilize the HC-SR04's speeds. It was determined that a fixed distance from the fan would be implemented in software, as it would be much faster to tell whether a signal is greater than or less than this distance. The author found that the best compromise between speed and usability for the nominal distance would be around 10cm, which means that the threshold time would be:

$$\text{Threshold} = \frac{2 \times 0.1\ \text{m}}{343\ \text{m/sec}} = 583.09\mu\text{s}$$

By comparing the echo time to this number, we can quickly determine whether the echo traveled more or less than 10cm. Additionally, the timeout threshold (meaning the time in which a trigger signal has “expired” is dramatically shortened to 3 times the threshold value, meaning that a LOW reading will only be measured between 10 cm and 30 cm (whereas a HIGH reading would be 10cm and below).

These changes were implemented as follows:

```
#define THRESHOLD_DISTANCE 10
#define THRESHOLD_TIME_MICROS (THRESHOLD_DISTANCE * 2 / (0.0343))
#define THRESHOLD_TIMEOUT (THRESHOLD_DISTANCE * 6 / (0.0343))
```

Figure 9: Thresholds, software lines 6-8

```
void onRecievedHCSR04(long echoTime){
 onBlade = echoTime < THRESHOLD_TIME_MICROS;
 totalCount += 1;

 if (onBlade && !prevOnBlade) riseCount += 1;
 else if (!onBlade && prevOnBlade) fallCount += 1;

 prevOnBlade = onBlade;
}
```

Figure 10: onReceivedHCSR04, software lines 60-67

With the ability to read rising and falling edges at a fast rate, converting to RPM was simple. Additionally, the theoretical max RPM can be derived from the value of totalCount (shown above). The following function is called every second:

```
void everySecond(unsigned long currentMillis){
   unsigned long time_elapsed_seconds = (currentMillis - previousMillis)/1000;
   previousMillis = currentMillis; // Update the last time

   // Action to perform after 1 second
   Serial.print("Current RPM: ");
   Serial.print(riseCount * 60 / time_elapsed_seconds);
   Serial.print(". Theoretical Max RPM: ");
   Serial.println(totalCount * 60 / time_elapsed_seconds);
```

```
    riseCount = 0;
    fallCount = 0;
    totalCount = 0;
}
```

Figure 11: everySecond, software lines 45-58

To test whether this software is viable for the project, some initial testing for the theoretical max fan speed needed to be done. In the following data (Figure 12), the theoretical max fan speed simply represents how many times the software could poll and read a distance measurement per minute.

Theoretical Max Fan Speed (RPM)
6840
6360
7620
6120
6780

Figure 12: Theoretical Max RPM Data (Appendix D)

In initial testing, without the use of a fan, the decrease in threshold and timeout clearly allowed for much higher possible RPM speeds for the resulting measured RPM and theoretical max RPM. Considering that there are 6 blades on the case fans in both personal desktop computers, this should be more than enough to measure up to 1000 RPM.

Appendix A includes the full software implementation of this custom library, including the pulse of the trigger pin and the handling of echo time.

BMP280

The BMP280 did not require any extraneous digital logic implementation, as the BMP280 library (BMP280.h) provided sufficient enough functionality of the BMP280.

```
// BMP280 Configuration
BMP280 bmp280;
uint32_t pressure;
float temperature;
```

Figure 13: BMP280 Configuration, lines 22-25

```
  // BMP280
  pressure = bmp280.getPressure();
  temperature = bmp280.getTemperature() * (9/5) + 32;

  Serial.print("Pressure: ");
  Serial.print(pressure);

  Serial.print(". Temperature: ");
  Serial.println(temperature);
```

Figure 14: BMP280 Measurement, lines 61-69

Photodiode and Kalman Filtering

Regarding the amplified photodiode ADC measurement, the signal is moderately noisy, even with a steady environment with no changes to dust concentration (see Figure 8). The filter was implemented in software by first recording some initial guesses for state, variance, noise, and noise variance:

```
// Kalman Filter Variables
float x_est = 1284.65; // Initial estimate of state (mean of data)
float P = 85.8;        // Initial estimate of covariance (variance of data)
const float Q = 1.0;   // Process noise covariance (tunable)
const float R = 85.8;  // Measurement noise covariance (from data)
float K;               // Kalman Gain
float z;               // Measurement input
```

Figure 15: Kalman Filter Variables

Then, with these initial values, the following Kalman filter is applied to the ADC reading of the dust concentration:

```
float kalmanFilter(float dustADC){
 float x_pred = x_est;  // Predicted state (A = 1)
 float P_pred = P + Q;  // Predicted error covariance
 // Update step
 K = P_pred / (P_pred + R);          // Kalman gain
 x_est = x_pred + K * (dustADC - x_pred);  // Updated state estimate
 P = (1 - K) * P_pred;               // Updated error covariance
 // Output the filtered value
 return x_est;
}
```

Figure 16: Kalman Filter Function

The Kalman filter is a recursive algorithm designed to estimate the state of a system from noisy measurements. It uses a two-step process — prediction and update. Because it is a recursive algorithm, it will dynamically adjust given each new data point at the expense of a delay.

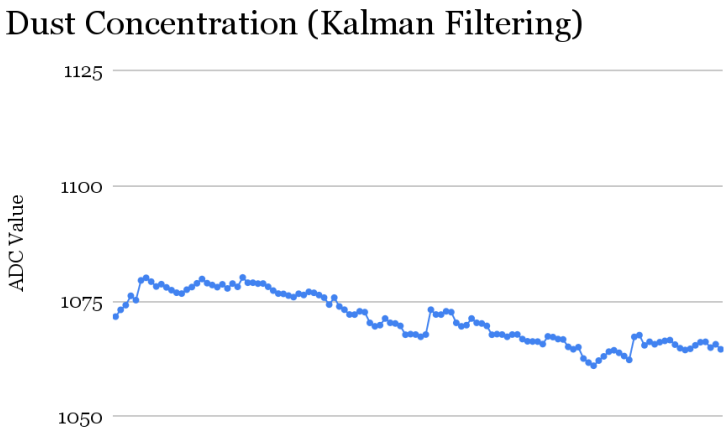


Figure 17: Filtering Applied

After implementing the Kalman filter function, the previously-noisy signal (shown in Figure 8) from the amplified photodiode voltage is now much more smooth.

Internet of Things

Four measurements are currently being uploaded to ThingSpeak — temperature, pressure, fan speed, and dust concentration voltage.

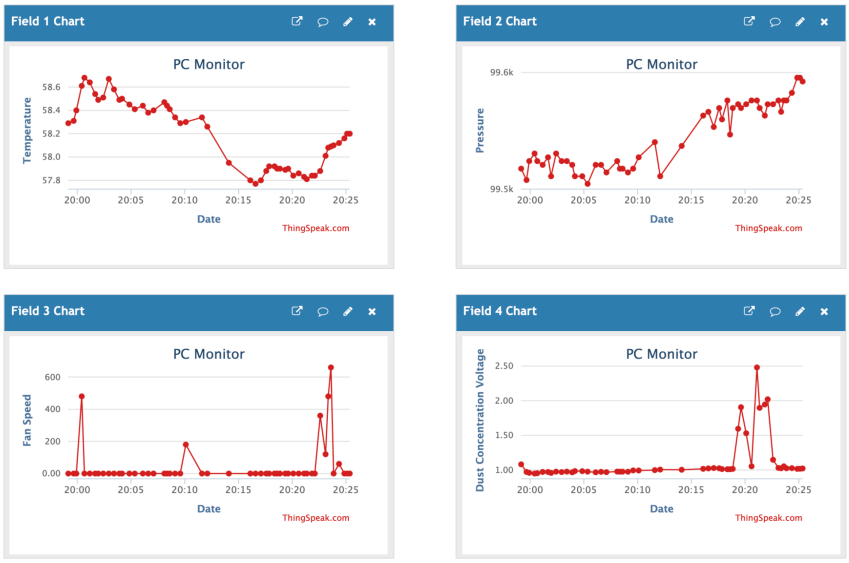


Figure 18: ThingSpeak, 9/24 8:37PM

For the link to the ThingSpeak, see Appendix C. For the full software implementation of ThingSpeak and WiFi connectivity, see Appendix A.

Experimental Results

Regarding experimental results, two videos were recorded — one focusing on fan speed and the other focusing on dust concentration.

RPM Test Video: Link

The RPM test video focuses on measuring fan speed, but also measures the effects of air flow on pressure. In the video, the HC-SR04 is initially directed towards a home fan spinning at ~500 RPM, and directed away from the fan at the end.

Dust Test Video: Link

The dust test video focuses on the measurement of IR light, representing dust concentration, and also shows the device working as a whole, additionally including temperature.

The results from the video and other measurements are as follows:

Field	Measured Value	Expected Value
Temperature, Rm 1450 (°F)	57.17	69.0
Pressure, No Air Flow (Pa)	99399 ± 2	Nominal, (101325)
Pressure, Air Flow (Pa)	99399 ± 2	Less than Nominal, (<99399 ± 2)
Fan Speed (RPM)	1370 ± 210	1500 (3 blades at 500 RPM)
Theoretical Max Speed (RPM)	6350 ± 1670	6870 ± 750 (from initial testing)
Dust Concentration Voltage, Minimum (V)	1.01	0.98 (originally 0.3)
Dust Concentration Voltage, Maximum (V)	2.70	2.76 (originally 3)

Figure 19: Experimental Results

The voltage fluctuates from 1.01V to 2.01V in the video and typically reaches up to 2.70V with direct IR light. The temperature measurement was 57.17 °F, and it successfully responds to

changes in temperature. The measured speed ranged from 1260 RPM to 1680 RPM. The pressure remained at a range of 99397 Pa to 99401 Pa despite the loss of air flow.

Analysis of Results

Regarding the temperature measurement of the BMP280, it consistently measured a temperature that was much lower than the actual temperature. Using BMP280s from other groups in my design provided much more accurate results, leading me to believe that my BMP280's temperature sensor was faulty. Regardless, the temperature still responds to changes in temperature correctly, which is important for application in this system. For future iterations, a transfer function between the measured and actual temperature can be done to provide the user with a much more accurate measurement. However, the temperature sensor in the motherboard of the PC can be referenced for the actual temperature reading for this iteration.

The pressure measurement to represent air flow was by far the least successful measurement of the project. The idea that the difference between the nominal and current air pressure could give an idea of air flow relies on the idea that these differences can be measured. However, most likely due to the low resolution and noise of the BMP280's pressure sensor, these differences were not able to be measured (see RPM Test Video), and, therefore, air flow could not be determined. In future iterations, it might be possible to apply further signal processing to the BMP280 in hardware and write a custom software package to measure the amplified signal (similar to the HC-SR04 section of Digital Logic Implementation), however the BMP280 is not meant for this application. Therefore, it might be more significant to use a different sensor to measure air flow.

Regarding fan speed, the results were fairly accurate, with the expected measurement of 1500 RPM being measured most of the time (see RPM Test Video); however, the range of measurements is quite large despite measuring a constant RPM. This is almost certainly due to the software implementation of the HC-SR04, where the RPM is being measured every second by counting the number of rising edges and multiplying by 60 (and dividing by the time elapsed). Perhaps changing the time elapsed would allow for more accurate values, since one or two missed rising edges causes the error to be hundreds of RPM. Increasing the time elapsed would allow these missed edges to be less impactful, at the expense of a less responsive measurement. In future iterations, it would perhaps be best to apply software filtering to the RPM measurement and treat the missed rising edges as noise, rather than sacrificing responsiveness. However, the goal of the custom library was certainly met, as the HC-SR04 proved to be capable of sampling at a fast enough rate to measure up to ~7000 RPM (theoretical max fan speed), and, if the HC-SR04 were to be used for this application again, future iterations should certainly build on this approach.

The dust concentration voltage was particularly hard to implement, but the experimental data show promising results. The amplification of a voltage that had a 0.04V range proved to be challenging. The following are the steps that were taken to achieve this:

1. For the first implementation, the OPO7CP was used to amplify the signal. However, it was necessary to also subtract a 0.54V voltage from the input voltage in order to properly scale the input into a large range. It was thought that the OPO7CP's offset pins (pins 1 and 8) could be used for this. However, those pins are for much smaller offsets than 0.54V, which meant a differential amplifier was needed in order to properly subtract this voltage.
2. For my second implementation, the goal was to only use the DC supply of the ESP32, rather than an external power supply, so the INA121P instrumentation amplifier was connected with 0V and 5V as Vcc inputs. However, any input voltages less than 1V were not being amplified, and the gain was not close to the expected gain. Therefore, the resulting signal, with it being nonlinear, was essentially unusable. See Appendix F for this data.
3. Because it was thought that the issue from the second implementation was due to saturation at the lower Vcc, the goal now was to amplify the signal twice — once through the OPO7CP and the next through the INA121P. The idea was that the OPO7CP could amplify the input to be in a usable range for the INA121P. However, while building the circuit, the author realized that this implementation would, for one, probably not work, due to the fact that gain was still incorrect even after an input was higher than 1V, and would also be too complicated for this simple application.
4. For the final implementation, the DC power supply was used for a -5V voltage on the Vcc- pin of the INA121P. Once connected, the output was completely as intended, where the input range of 0.54V to 0.58V was amplified to be 0.3V to 3V. Appendix F goes over the comparison between the second and fourth implementations of the amplification.

Once amplifying the signal to a usable range, the dust concentration measurement seemed to work very well in theory, and tests were executed using a second IR emitter that would simulate scattered IR light. Perhaps the inclusion of a battery to achieve a negative voltage would have been more sufficient in order to test the dust concentration voltage in dusty environments. However, the results seem very promising and achieved the intended output.

Conclusion

In the Experimental Results section, it was discussed the many improvements that could be made to a second revision of this device — here are the points that are most significant for the next revision:

- Create a transfer function to measure accurate temperature readings from a faulty BMP280, or use a BMP280 that is not faulty

- Use a designated air flow sensor such as the FS3000-1015 by SparkFun rather than a pressure sensor.
- Implement noise filtering for the fan speed measurement due to missed rising edges on the HC-SR04
- Include a battery to achieve a negative voltage for the INA121P.

From this project, I learned the most about the general process of implementing and testing a design, and it is not necessarily important whether the results of these tests are successful or not. Even with this design, there are many flaws, but it allowed me to make well informed recommendations towards a second revision. Finally, I learned that, personally, thinking about implementing ideas means nothing compared to taking action towards actually implementing them, since most of the time my ideas need to be completely reworked once testing them.

Human References

BMP280
https://www.adafruit.com/product/2651

HC-SR04
https://www.adafruit.com/product/3942

VL53L0X
https://www.adafruit.com/product/3317

Infrared LED
https://www.sunledusa.com/products/spec/XTNI12BF.pdf

Infrared Photodiode
https://www.onsemi.com/pdf/datasheet/qsd2030f-d.pdf

Kalman Filtering
https://www.kalmanfilter.net/default.aspx

OPO7CP
https://www.ti.com/lit/ds/symlink/op07c.pdf?ts=1732625494472&ref_url=https%253A%252F%252Fwww.google.de%252F

INA121P
https://www.ti.com/lit/ds/symlink/ina121.pdf?ts=1732629902199

FS3000-1015
https://www.sparkfun.com/products/18768?gad_source=1&gbraid=0AAAADsj4ER2y1PDU1DlKowIGeSpYS4rb&gclid=Cj0KCQiAgJa6BhCOARIsAMiL7V8BzDMjKnFWi1JUvNXd3IgGQ8sQB9S5uuayz6d_VV5jloBxLVfBqAkaAmu-EALw_wcB

AI References

OpenAI. (2024). *ChatGPT* (November 4 version) [Large language model]. OpenAI. https://openai.com

- Initially used to “bounce” ideas off of and see whether certain ideas were viable
- Debugging code
- Tutorials on packages
- Helped a lot with getting WiFi to work in software.
- Helped with amplification troubleshooting

Appendix

Appendix A: Full Code

```
#include <WiFi.h>
#include <ThingSpeak.h>
#include "Arduino.h"
#include <BMP280.h>

// Configuration
#define DUST_PIN 33
#define TRIGGER_PIN 2
#define ECHO_PIN 4
#define THRESHOLD_DISTANCE 10
#define THRESHOLD_TIME_MICROS (THRESHOLD_DISTANCE * 2 / (0.0343))
#define THRESHOLD_TIMEOUT (THRESHOLD_DISTANCE * 6 / (0.0343))

// One Second Loop
unsigned long previousMillisOne = 0;
unsigned long previousMillisFifteen = 0;

// HC-SR04 Configuration
bool onBlade = false;
bool prevOnBlade = false;
int riseCount = 0;
int fallCount = 0;
int totalCount = 0;
float fanSpeed = 0;

// BMP280 Configuration
BMP280 bmp280;
uint32_t pressure;
float temperature;

// Photodiode for Dust Concentration
```

```
float dustVoltage;

// Kalman Filter Variables
float x_est = 1284.65; // Initial estimate of state (mean of data)
float P = 85.8;        // Initial estimate of covariance (variance of data)
const float Q = 1.0;   // Process noise covariance (tunable)
const float R = 85.8;  // Measurement noise covariance (from data)
float K;               // Kalman Gain
float z;               // Measurement input

// WiFi Connection and ThingSpeak
const char* ssid = "ioT_Umair";       // Replace with your Wi-Fi SSID
const char* password = "123456789";   // Replace with your Wi-Fi password
WiFiClient client;

unsigned long channelID = 2760876;    // Replace with your ThingSpeak Channel ID
const char* apiKey = "2AI6G5ZS234XDZAM"; // Replace with your ThingSpeak Write API Key

unsigned long lastUploadTime = 0;
const unsigned long uploadInterval = 15000; // Upload interval in milliseconds (15 seconds)

void setup() {
 pinMode(TRIGGER_PIN, OUTPUT);
 pinMode(ECHO_PIN, INPUT);
 pinMode(DUST_PIN, INPUT);
 Serial.begin(9600);

 Wire.begin(); //Join I2C bus
 bmp280.begin();

 // Connect to Wi-Fi
 Serial.print("Connecting to Wi-Fi");
 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
   delay(1000);
   Serial.print(".");
 }
 Serial.println("\nWi-Fi connected!");

 // Initialize ThingSpeak
 ThingSpeak.begin(client);
}

void loop() {
 // HC-SR04 ROUTINE

 // send pulse from trigger pin
 sendPulseHCSR04();

 // Measure echo time
 long echoTime = pulseIn(ECHO_PIN, HIGH, THRESHOLD_TIMEOUT);

 // Check if the time is below the threshold, and increment counters when received
 if (echoTime > 1) onRecievedHCSR04(echoTime);


 // EVERY SECOND
 unsigned long currentMillis = millis();
 if (currentMillis - previousMillisOne >= 1000) everySecond(currentMillis);

 // EVERY 15 SECONDS
 if (currentMillis - previousMillisFifteen >= 15000) everyFifteenSeconds(currentMillis);
}
```

```
void everySecond(unsigned long currentMillis){
 unsigned long time_elapsed_seconds = (currentMillis - previousMillisOne)/1000;
 previousMillisOne = currentMillis; // Update the last time

 // Action to perform after 1 second
  // BMP280
 pressure = bmp280.getPressure();
 temperature = bmp280.getTemperature() * (9/5) + 32;

 Serial.print("Pressure: ");
 Serial.print(pressure);

 Serial.print(". Temperature: ");
 Serial.println(temperature);

 // HC-SR04
 fanSpeed = riseCount * 60 / time_elapsed_seconds;
 Serial.print("Current RPM: ");
 Serial.print(fanSpeed);
 Serial.print(". Theoretical Max RPM: ");
 Serial.println(totalCount * 60 / time_elapsed_seconds);

 riseCount = 0;
 fallCount = 0;
 totalCount = 0;

 // Photodiode
 dustVoltage = kalmanFilter(analogRead(DUST_PIN));
 dustVoltage = (dustVoltage * 3 / 4095) + 0.28;
 Serial.print("Dust Voltage: ");
 Serial.print(dustVoltage);
 Serial.println("V");
 Serial.println("--------------------");
}

void everyFifteenSeconds(unsigned long currentMillis){
 previousMillisFifteen = currentMillis; // Update the last time

 // Action to perform after 15 seconds
 uploadToThingSpeak(temperature, pressure, fanSpeed, dustVoltage);
}

void onRecievedHCSR04(long echoTime){
 onBlade = echoTime < THRESHOLD_TIME_MICROS;
 totalCount += 1;
 if (onBlade && !prevOnBlade) riseCount += 1;
 else if (!onBlade && prevOnBlade) fallCount += 1;

 prevOnBlade = onBlade;
}

void sendPulseHCSR04(){
 digitalWrite(TRIGGER_PIN, LOW);
 delayMicroseconds(1);
 digitalWrite(TRIGGER_PIN, HIGH);
 delayMicroseconds(4);
 digitalWrite(TRIGGER_PIN, LOW);
}

float kalmanFilter(float dustADC){
 float x_pred = x_est;  // Predicted state (A = 1)
 float P_pred = P + Q;  // Predicted error covariance
```

```
 // Update step
 K = P_pred / (P_pred + R);          // Kalman gain
 x_est = x_pred + K * (dustADC - x_pred);  // Updated state estimate
 P = (1 - K) * P_pred;               // Updated error covariance

 // Output the filtered value
 return x_est;
}

void uploadToThingSpeak(float temperature_1, float pressure_1, float fanSpeed_1, float dustConcentration_1) {
 ThingSpeak.setField(1, temperature_1);
 ThingSpeak.setField(2, pressure_1);
 ThingSpeak.setField(3, fanSpeed_1);
 ThingSpeak.setField(4, dustConcentration_1);

 int response = ThingSpeak.writeFields(channelID, apiKey);
 if (response == 200) {
   Serial.println("Data sent to ThingSpeak successfully.");
 } else {
   Serial.print("Failed to send data to ThingSpeak. Response: ");
   Serial.println(response);
 }
}
```

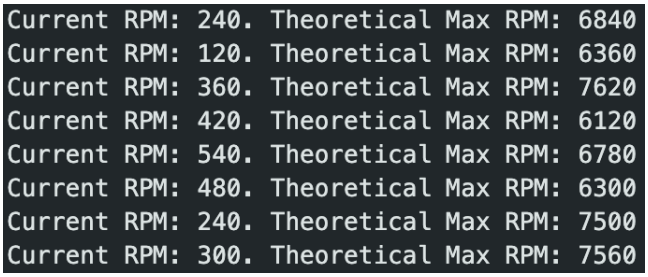
Appendix B: General Safety Checklist

General Safety Checklist

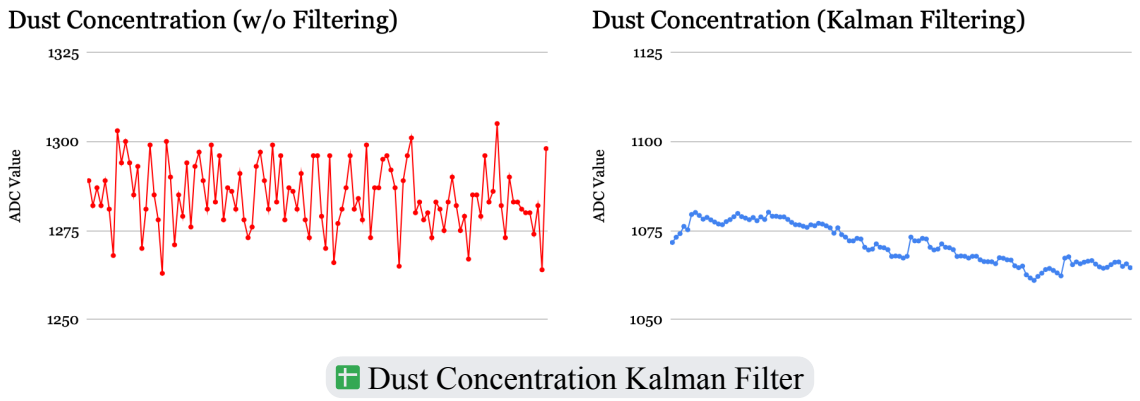
Appendix C: ThinkSpeak Link

https://thingspeak.mathworks.com/channels/2760876

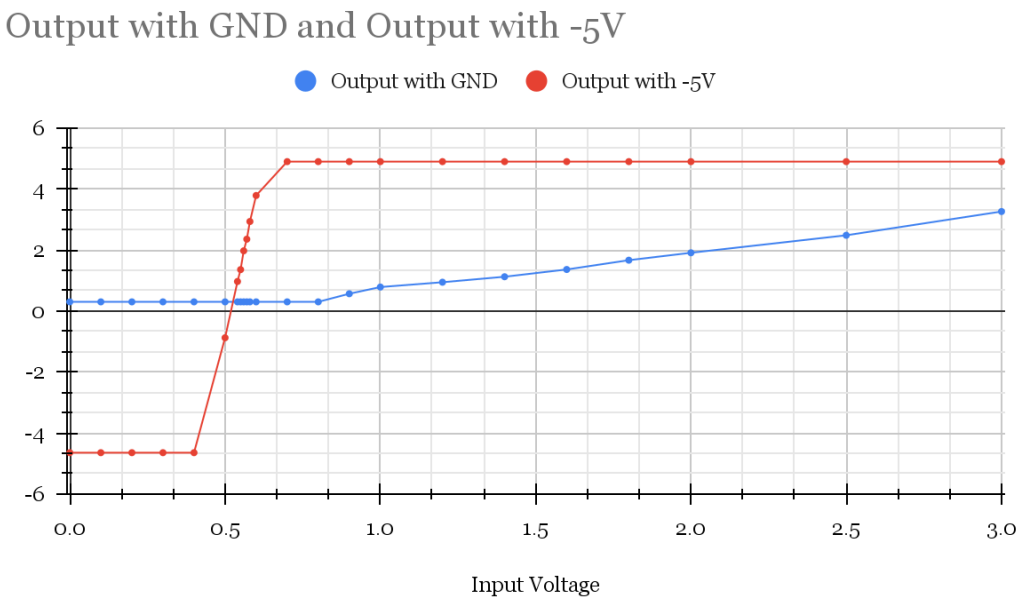
Appendix D: Max RPM Data Screenshot



Appendix E: Kalman Filtering Data



Appendix F:



Amplification Data